SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool *(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 3 **Total Marks:50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

*I **K.YOGANAND/192525173**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation**, including **lexical, syntax, semantic, logical, and runtime errors**.

Given Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
int i, sum = 0;     for(i = 0; i
<= n; i++) {         sum = sum
+ arr[i];
   }
   return sum;
}

int main() {     int arr[5]
= {1,2,3,4,5}     int total;

   total = sumArray(arr);

   if(total == 15) {
printf("Correct Sum\n")
   }

   int a = 10, b = 0;
int c = a / b;

   int *p;
   *p = 20;

   return 0;
}
```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all the errors present in the program. For each error:
2. Specify the line number
3. Clearly describe the nature of the error

(b) Classification of Errors (Understanding + Analysis)

1. Classify each identified error into the appropriate compiler phase:
2. Lexical Errors
3. Syntax Errors
4. Semantic Errors
5. Logical Errors
6. Runtime Errors

(c) Compilation Behavior (Evaluation Level)

1. Analyze which errors will be detected during compilation and which will only appear during execution.
2. Explain how early-stage errors (lexical/syntax) may prevent detection of laterstage errors.

(d) Code Correction (Application Level)

1. Rewrite the corrected version of the program such that:
2. All errors are removed
3. The program executes correctly
4. Proper syntax and semantics are maintained

(e) Impact Analysis (Evaluation Level)

1. Discuss the impact of at least three errors on program execution.
2. Explain how incorrect constructs like improper pointer usage, division by zero, and infinite loops affect system behavior.

(f) Advanced Compiler Perspective (Create Level)

1. Suggest how a compiler can be improved to:
2. Provide better error messages
3. Perform early detection of runtime errors
4. Propose at least two enhancements in compiler design for efficient debugging.

Q2.The following C program contains **multiple lexical errors** introduced intentionally. These errors occur due to **invalid tokens, incorrect identifiers, malformed constants, and illegal symbols**, which are typically detected during the **lexical analysis phase** of a compiler.Carefully examine the program and answer the questions given below. #include <stdio.h>

```
int main() {     int
1num = 100;     float
rate@ = 25.5;     char
ch = 'abc';
   int total = 50$20;
int num#1 = 30;
   int %count = 40;

   printf("Welcome to Compiler Design);

   int x = 09;
int y = 0xZ12;
   int z = 12.3.4;

   char str1[] = "Hello;
   char str2[] = "World";

   int _valid = 10;
   int invalid-id = 20;

   float value = 5..6;
   int data = 10@5;

   char c = 'xy';
   int 3value = 60;

   printf("Value: %d\n", total);
```

```
    return 0;
}
```

Tasks

(a) Error Identification

i) Identify all lexical errors in the program.

ii) List the line number and the invalid token.

(b) Error Classification

i) For each error, explain why it is considered a lexical error.

ii) Specify which rule of token formation is violated:

iii) Identifier rules iv) Constant formation

v) String/character literal rules vi) Operator/symbol rules

(c) Token Correction

i)Rewrite each invalid token into a valid token. ii)Example:

2value → value2

(d) Compiler Behavior Analysis

i) Explain how a lexical analyzer (scanner) processes such errors.

ii)Discuss whether the compiler:

iii)Stops immediately

iv)Continues scanning using error recovery

(e) Advanced Thinking

i) Design a simple rule (or DFA idea) to detect invalid identifiers starting with digits. ii) Suggest how error messages can be improved for beginners.

Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains **multiple errors distributed across different phases of a compiler**, including:

A. Lexical Errors

B. Syntax Errors

C. Semantic Errors

D. Intermediate Code Generation Issues (logic/precedence)

E. Code Optimization Issues (redundancy/inefficiency)

F. Runtime / Target Code Errors

Carefully analyze the program and answer the questions below.

```
#include <stdio.h>

int divide(int a, int b) {
return a / b
}
```

```
int main() {     int
#value = 20;
float num = 15.5
int result;

   result = divide(10);

   if(result = 5) {
printf("Result is 5\n")
   }

   int x = 10;
int y = 0;     int
z = x / y;

   int arr[4] = {1,2,3,4};
printf("%d\n", arr[4]);

   int *ptr;
   *ptr = 25;
    int total;     total = x + y
* result + 0;

   while(x < 20) {
      printf("%d\n", x);
   }

   return 0;
}
```

Do the following Tasks

(a) Identify Errors (Analysis Level)

I. Identify **all errors** in the program.

II. Mention **line number + error description**.

(b) Classify by Compiler Phase Classify each error into:

I. Lexical (invalid symbols like @num)

II. Syntax (missing semicolon, brackets)

III. Semantic (function argument mismatch)

IV. Intermediate Code Issues (wrong expression handling/precedence) V.

Optimization Issues (redundant + 0)

VI. Runtime Errors (division by zero, invalid pointer access)

(c) Compilation vs Execution (Evaluation)

I. Which errors are detected at **compile time**?

II. Which errors appear only during **execution**?

III. Explain why some errors prevent further compilation.

(d) Correct the Program (Application) I.

Rewrite the **fully corrected program**.

II. Ensure:

A. Proper syntax
B. Correct semantics
C. Safe execution

(e) Deep Analysis (Evaluation)

I. Explain how **incorrect pointer usage** affects runtime.

II. Analyze how **array bounds violation** impacts memory.

III. Discuss how **operator precedence errors** affect intermediate code.

(f) Advanced Task (Create Level)

I. Suggest **2 improvements** in compiler design to:

A. Detect runtime errors earlier
B. Improve debugging messages

II. Propose how **optimization phase** can remove redundant expressions.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)

Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

\

Q1. ERROR ANALYSIS TASK

Given Program #include

<stdio.h> int sumArray(int

arr[], int n) { int i, sum = 0;

for(i = 0; i <= n; i++) {

sum = sum + arr[i];

}

return sum; } int main() {

int arr[5] = {1,2,3,4,5} int

total; total =

sumArray(arr); if(total ==

15) { printf("Correct

Sum\n")

}

int a = 10, b = 0;

int c = a / b; int

*p; *p = 20;

return 0;

}

(a) Error Identification

Line	Error	Description
5	Logical error	i <= n causes access to arr[n], which is outside the valid range when n = 5.
12	Syntax error	Missing semicolon after array initialization.
15	Semantic error	sumArray() requires two arguments but only arr is supplied.
18	Syntax error	Missing semicolon after printf().
22	Runtime error	a / b performs integer division by zero because b = 0.
25	Runtime/undefined behavior	p is uninitialized and is dereferenced.

Line 5 explanation The

array is:

int arr[5] = {1,2,3,4,5}; Valid

indexes are:

0, 1, 2, 3, 4

But the loop executes when:

i = 5

and accesses:

arr[5] which is

invalid.

Correct:

for(i = 0; i < n; i++)

(b) Classification of Errors

Line	Error	Classification	Phase
5	i <= n	Logical	Analysis/Execution
12	Missing ;	Syntax	Syntax Analysis
15	Missing argument	Semantic	Semantic Analysis
18	Missing ;	Syntax	Syntax Analysis
22	Division by zero	Runtime	Execution
25	Invalid pointer dereference	Runtime/Undefined behavior	Execution

Lexical Errors

There are **no intentional lexical errors** in Q1.

All characters and tokens are valid C tokens.

(c) Compilation Behavior

Errors detected during compilation

The following can be detected before execution:

Line 12 int arr[5] = {1,2,3,4,5}

The compiler expects ;.

Line 18 printf("Correct

Sum\n") Again, ; is

missing. **Line 15** total

= sumArray(arr); The

function declaration is:

int sumArray(int arr[],

int n) It requires: array

+ size

Correct:

total = sumArray(arr, 5);

Errors appearing during execution Line

22

int b = 0; int

c = a / b;

This causes division by zero.

Line 25 int *p; *p = 20; p does

not point to a valid object.

Line 5

The array-bound problem can occur during execution because normal C compilation does not automatically perform runtime bounds checking. **Why early errors prevent later error detection** Compiler processing is approximately:

Source Code

↓

Lexical Analysis

↓

Syntax Analysis

↓

Semantic Analysis

↓

Intermediate Code

↓

Optimization

↓

Code Generation

↓

Execution

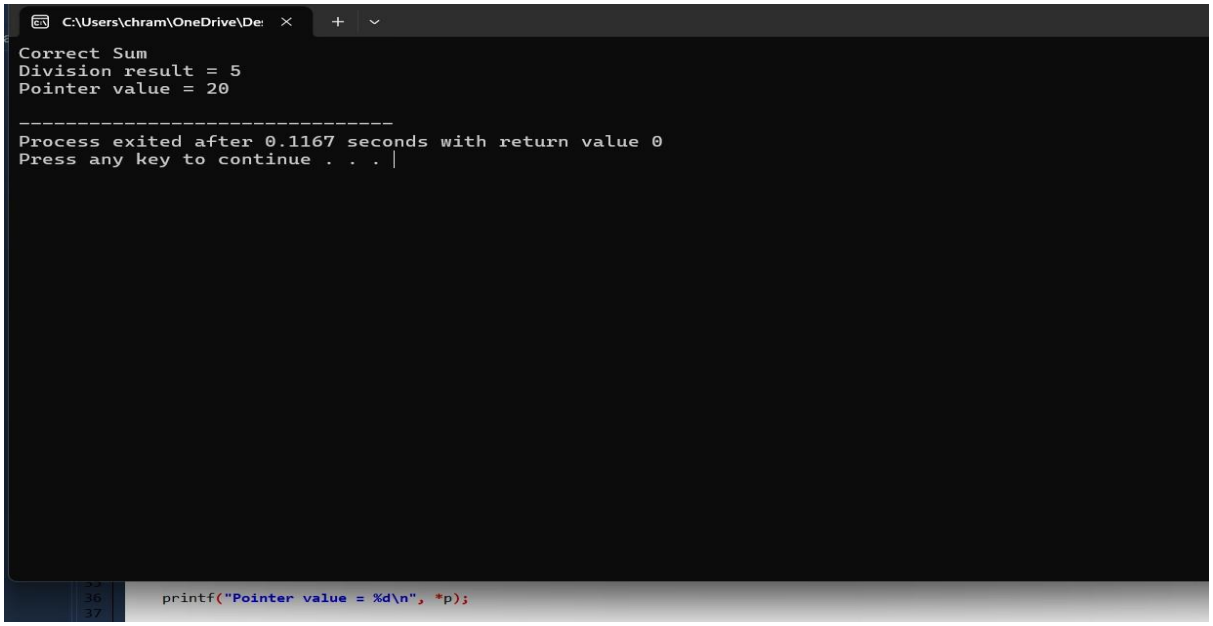
If a syntax error occurs early, later phases may not receive a valid representation of that part of the program.

For example, a missing ; can cause the parser to misunderstand subsequent statements and generate **cascading errors**.

(d) Corrected Program

```
#include <stdio.h> int
sumArray(int arr[], int n) {
int i, sum = 0;     for (i = 0; i <
n; i++) {         sum = sum +
arr[i];
   }
   return sum;
} int main()
{
   int arr[5] = {1, 2, 3, 4, 5};
int total;
   total = sumArray(arr, 5);
if (total == 15) {
printf("Correct Sum\n");
   }
   int a = 10, b = 2;
if (b != 0) {
int c = a / b;
      printf("Division result: %d\n", c);
   } else {
      printf("Cannot divide by zero\n");
   }     int value;
int *p = &value;
*p = 20;
   printf("Pointer value: %d\n", *p);
return 0; }
```

Output



Correct Sum

Division result: 5

Pointer value: 20

(e) Impact Analysis 1.

Improper pointer usage

Original: int *p;

*p = 20;

The pointer contains an indeterminate address.

Therefore, dereferencing it can access an invalid memory location.

Possible result:

Segmentation fault or other

undefined behavior.

Correct: int

value; int *p =

&value; *p =

20;

2. Division by zero Original: int a =

10, b = 0; int c = a / b;

Division by zero is undefined behavior in C.

It can cause a runtime exception or program termination.

Safe approach:

if (b != 0)

c = a / b;

else

printf("Cannot divide by zero");

3. Array boundary violation

Original:

for(i = 0; i <= n; i++) For n

= 5, the loop accesses:

arr[0] arr[1]

arr[2] arr[3]

arr[4] arr[5] ←

invalid

This can read unrelated memory and cause undefined behavior.

4. Infinite loop

An incorrectly designed loop can continue indefinitely.

For example: while(i

< 10) {

printf("%d", i);

} if i is never

incremented.

This consumes CPU resources and prevents the program from reaching later statements.

(f) Advanced Compiler Perspective

Improvement 1 – Static analysis

A compiler can analyze the source code before execution to detect:

- Possible division by zero
- Uninitialized variables
- Uninitialized pointers
- Out-of-bounds array access
- Unreachable code
- Suspicious conditions Example:

int b = 0; int c = a / b;

The compiler can issue:

Warning: possible division by zero.

Improvement 2 – Better diagnostics

Instead of: error: expected ';' the

compiler could report:

Error at line 12:

Missing ';' after array initialization.

Current:

int arr[5] = {1,2,3,4,5} Suggested:

int arr[5] = {1,2,3,4,5};

This is much more useful for beginners.

Q2. LEXICAL ERROR ANALYSIS

(a) Error Identification

Line	**Invalid token**	**Error**
4	1num	Identifier starts with a digit
5	rate@	@ is not allowed in an identifier
6	'abc'	Character literal contains multiple characters
7	50$20	$ is not a standard C arithmetic/operator token
8	num#1	# cannot appear in an identifier
9	%count	% cannot begin an identifier
11	"Welcome to Compiler Design)	Unterminated string literal
13	09	Invalid octal constant
14	0xZ12	Invalid hexadecimal constant
15	12.3.4	Malformed floating-point constant
17	"Hello	Unterminated string literal
21	invalid-id	- is an operator, not an identifier character
23	5..6	Malformed floating-point constant
24	10@5	@ is not a standard C operator
26	'xy'	Multi-character character constant
27	3value	Identifier starts with a digit

Technical note: In a real C compiler, some of these may be diagnosed during parsing rather than strictly by the scanner. For this assignment, they are classified according to the intended lexical-token-formation errors.

(b) Error Classification

1. Identifier rules

A C identifier can contain: A-Z

a-z

0-9

_ but the first character must be a letter or

_.

Therefore:

1num

3value are

invalid.

Correct: num1

value3

2. Constant formation

09

A leading 0 indicates an octal integer constant.

Valid octal digits are:

0 1 2 3 4 5 6 7

Therefore: 09

is invalid.

Correct:

9

0xZ12

A hexadecimal constant can contain:

0-9 A-F

a-f

Z is invalid.

Correct example:

0xA12

12.3.4

A floating-point constant cannot have two decimal points.

Correct:

12.34

3. String and character literal rules

'abc'

A normal character constant represents a single character:

'a'

not:

'abc'

Correct:

'a'

'xy'

Correct:

'x'

Unterminated strings Incorrect:

"Hello

Correct:

"Hello"

4. Operator/symbol rules Examples:

@

$

#

%

cannot be arbitrarily inserted into identifiers or expressions.

For example:

int num#1;

should be: int

num1;

(c) Token Corrections

Invalid	**Correct**
1num	num1
rate@	rate

'abc'	'a'
50$20	50 + 20
num#1	num1
%count	count
"Welcome to Compiler Design)	"Welcome to Compiler Design"
09	9
0xZ12	0xA12
12.3.4	12.34
"Hello	"Hello"
invalid-id	invalid_id
5..6	5.6
10@5	10 + 5
'xy'	'x'
3value	value3

(d) Compiler Behavior Analysis

The lexical analyzer reads characters and groups them into tokens.

For:

int value = 10;

tokens include: int

→ keyword value

→ identifier =

→ operator

10 → constant

; → delimiter For:

int 1num = 100; the scanner cannot form 1num as a

valid C identifier.

It reports an error and may attempt to recover.

Does the compiler stop immediately?

Usually, a compiler tries to report **multiple errors** rather than stopping after the first one.

Typical recovery:

Detect error

↓

Report error

↓

Skip invalid characters/token

↓

Find synchronization point

↓

Continue scanning

Synchronization points can include:

;

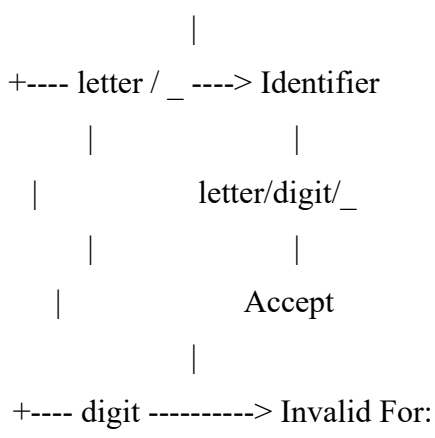
}

However, an unterminated string can cause significant confusion because the scanner may interpret following characters as part of the same string.

(e) Advanced Thinking

DFA for invalid identifiers beginning with digits

A valid identifier can be represented as: Start



For:

3value the first character is

a digit. Therefore:

3 → Invalid identifier start A

compiler can report:

Error: '3value' is not a valid identifier.

Identifiers cannot begin with a digit.

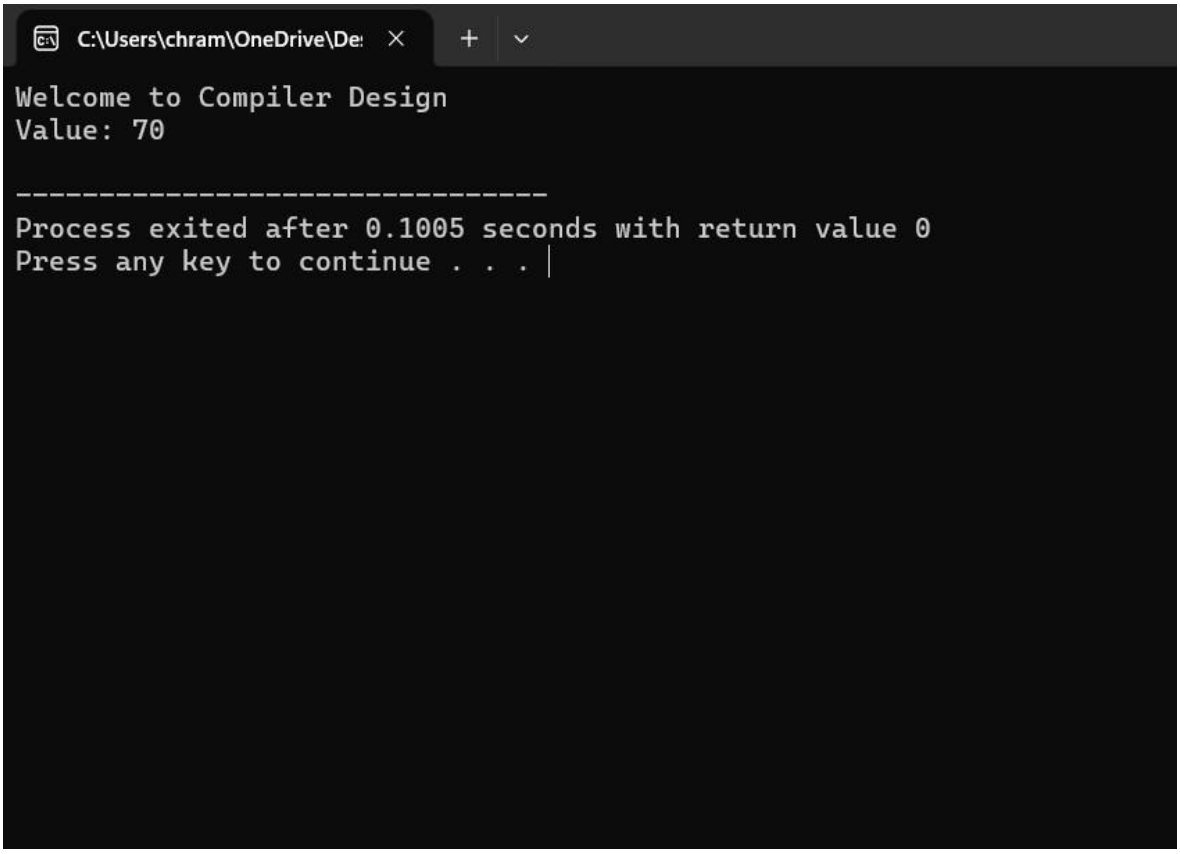
Suggestion: use 'value3'.

Corrected Code:

```
#include <stdio.h>
int main() {     int
num1 = 100;     float
rate = 25.5;     char
ch = 'a';     int total =
50 + 20;     int num2
= 30;     int count =
40;
   printf("Welcome to Compiler Design\n");
int x = 9;     int y = 0xA12;     float z =
12.34;     char str1[] = "Hello";     char str2[]
= "World";     int _valid = 10;     int
invalid_id = 20;     float value = 5.6;     int
data = 10 + 5;     char c = 'x';     int value3 =
60;

   printf("Value: %d\n", total);
return 0;
}
```

OUTPUT:



Q3. MULTI-PHASE ERROR ANALYSIS

(a) Error Identification

Line	Error	Classification
4	Missing ; after return a / b	Syntax
8	#value invalid identifier	Lexical/Syntax
9	Missing ;	Syntax
12	divide(10) has insufficient arguments	Semantic
14	result = 5 uses assignment instead of comparison	Logical
15	Missing ;	Syntax
20	Division by zero	Runtime
23	arr[4] outside valid range	Runtime/Undefined behavior
26	Uninitialized pointer dereference	Runtime/Undefined behavior
29	+ 0 is redundant	Optimization issue
31–33	x never changes	Logical/Infinite loop

(b) Classification by Compiler Phase

I. Lexical Error

Line 8: int #value = 20; #

is not part of a valid C

identifier.

Correct: int

value = 20;

II. Syntax Errors

Line 4 return a / b

Correct: return a / b;

Line 9 float num =

15.5 Correct: float

num = 15.5; **Line 15**

printf("Result is 5\n")

Correct:

printf("Result is 5\n");

III. Semantic Error

Line 12:

result = divide(10);

Function declaration: int

divide(int a, int b)

requires two parameters.

Correct:

result = divide(10, 2);

IV. Logical Error

Line 14:

if(result = 5)

= performs assignment.

It changes result to 5.

For comparison, use:

if(result == 5)

The original code can compile because assignment expressions are legal in C. Therefore this is mainly a **logical/programming error**.

V. Optimization

Issue Line 29:

total = x + y * result + 0;

Adding zero is redundant.

Compiler optimization can transform it into:

total = x + y * result;

This is an example of **algebraic simplification**.

VI. Runtime Errors

Line 20

int y = 0; int

z = x / y;

Division by zero. **Line**

23

int arr[4] = {1,2,3,4};

printf("%d\n", arr[4]);

Valid indexes: 0, 1, 2, 3

arr[4] is outside the array.

Line 26 int *ptr; *ptr =

25; ptr is uninitialized.

Dereferencing it results in undefined behavior.

Lines 31–33 while(x < 20) { printf("%d\n",

x);

}

x remains 10.

Therefore the condition remains true forever.

(c) Compilation vs Execution

Compile-time errors

These can be identified during compilation:

Line 4 → missing ;

Line 8 → invalid #

Line 9 → missing ;

Line 12 → incorrect function arguments

Line 15 → missing ;

Execution-time problems

These occur when the relevant statements execute:

Line 20 → division by zero

Line 23 → array out of bounds

Line 26 → invalid pointer dereference

Line 31 → infinite loop

Some of these are formally **undefined behavior**, so C does not guarantee exactly what will happen.

Why errors prevent further compilation

Compilation occurs in stages.

For example:

Lexical Analysis

↓

Syntax Analysis

↓

Semantic Analysis

↓

Intermediate Code

↓

Optimization

↓

Code Generation

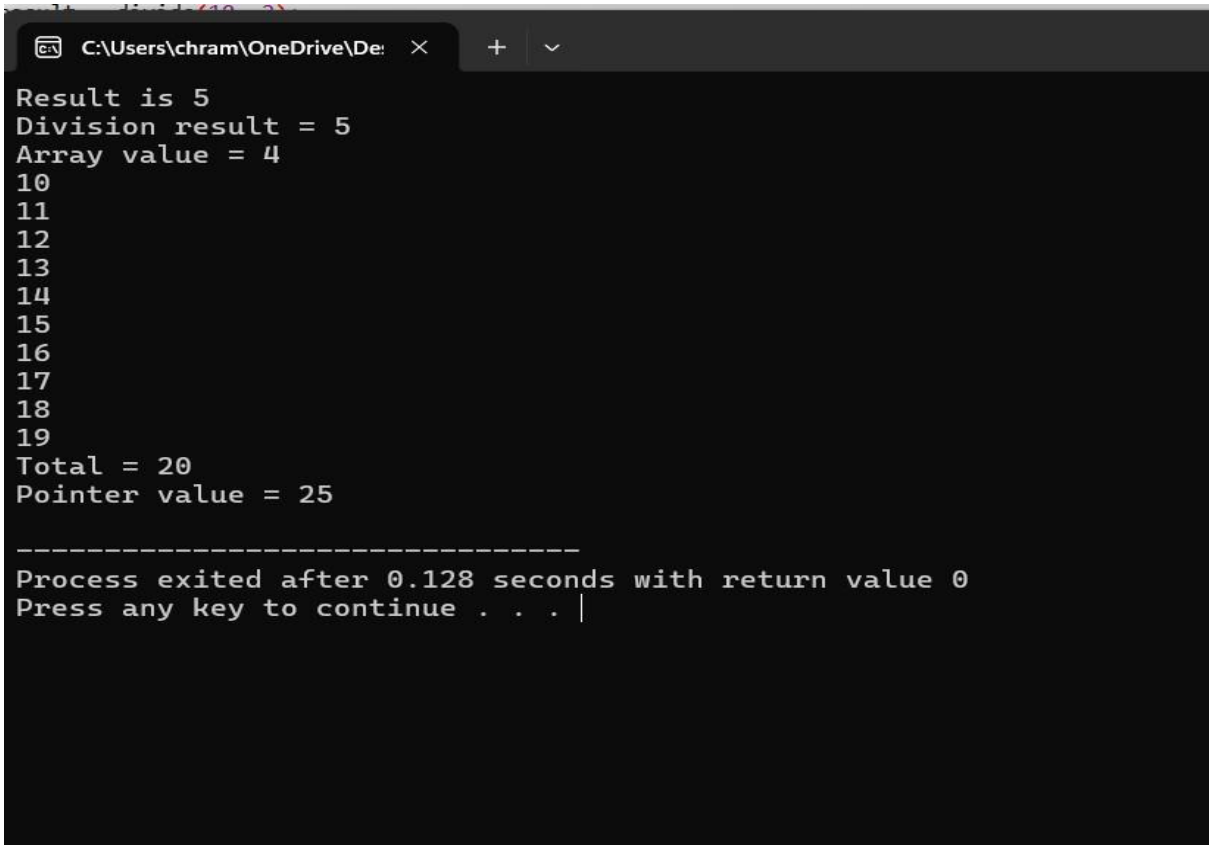
A serious syntax error can prevent the compiler from constructing the correct syntax tree. Therefore, later stages may not be able to process that portion of the program.

(d) Fully Corrected Program #include <stdio.h> int divide(int a, int b) { if (b == 0) { printf("Error: Division by zero\n"); return 0; } return a / b; } int main() { int value = 20; float num = 15.5f; int result; result = divide(10, 2); if (result == 5) { printf("Result is 5\n");

```
    }     int x = 10;     int y = 2;     int z =
x / y;     printf("Division result: %d\n",
z);     int arr[4] = {1, 2, 3, 4};     if (3 <
4) {         printf("Array value: %d\n",
arr[3]);
    }     int ptrValue;
int *ptr = &ptrValue;
*ptr = 25;     int total;
total = x + y * result;
while (x < 20) {
printf("%d\n", x);
x++;     }

    printf("Total = %d\n", total);
printf("Pointer value = %d\n", *ptr);
return 0; }
```

Output



(e) Deep Analysis

1. Pointer Usage

Original: int

*ptr;

*ptr = 25;

The declaration creates a pointer but does not give it a valid target.

Conceptually: ptr

→ ???

Therefore: *ptr = 25; tries to write to

an unknown address.

Correct method

int value; int *ptr

= &value;

*ptr = 25; Now: ptr →

value and the dereference is

valid. **2. Array Bounds**

Violation Given:

int arr[4] = {1,2,3,4}; The

valid indexes are:

arr[0] = 1

arr[1] = 2

arr[2] = 3

arr[3] = 4

But:

arr[4] is

invalid.

C does not automatically check array boundaries.

Possible consequences:

- Incorrect values
- Memory corruption
- Program crash
- Undefined behavior
- Potential security vulnerabilities

3. Operator Precedence

Expression: x + y *

result

Multiplication has higher precedence than addition.

Therefore: x +

(y * result) is

evaluated.

It is **not**:

(x + y) * result

For: x = 10 y = 2

result = 5 correct

calculation:

10 + (2 × 5)

= 10 + 10

= 20

Intermediate representation

The expression can be represented as:

+

/ \ x

*

/ \

y result This

preserves operator precedence.

4. Infinite Loop

Original: int x = 10;

while(x < 20) {

printf("%d\n", x);

}

The value of x never changes.

Therefore:

10 < 20 → TRUE

10 < 20 → TRUE

10 < 20 → TRUE

...

The loop never terminates.

Correct: while(x <

20) {

printf("%d\n", x);

x++; }

(f) Advanced Task

Improvement 1 – Static analysis

A compiler can perform static analysis to detect:

- Possible division by zero
- Uninitialized variables
- Uninitialized pointers
- Possible array-bound violations
- Suspicious assignments in conditions
- Unreachable code Example:

if(result = 5)

The compiler can provide:

Warning:

Assignment used as condition.

Did you mean '=='?

Improvement 2 – Runtime sanitizers

Compiler-generated instrumentation can detect memory-related problems during testing, such as:

Out-of-bounds access

Invalid pointer dereference

Use-after-free

Memory corruption

This allows programmers to find bugs before deployment.

Optimization Phase

The optimizer can use algebraic simplification.

Examples:

x + 0 → x

x - 0 → x x

* 1 → x x /

1 → x

Therefore:

total = x + y * result + 0; can

become:

total = x + y * result;

This reduces unnecessary operations and may improve execution efficiency.

Final Comparison of All Three Questions

Question	Main Focus	Important Errors
Q1	Multi-phase error analysis	Syntax, semantic, logical, runtime
Q2	Lexical analysis	Invalid identifiers, constants, symbols, literals
Q3	Complete compiler pipeline	Lexical, syntax, semantic, intermediate code, optimization, runtime